

Klasické metodiky

Karel Richta

Katedra elektrotechniky a informatiky
Vysoká škola polytechnická
Jihlava

© Karel Richta, 2016

Softwarové inženýrství, SWI
04/2016, Lekce 9

<https://elearning.vspj.cz/course/search.php?search=SWI>

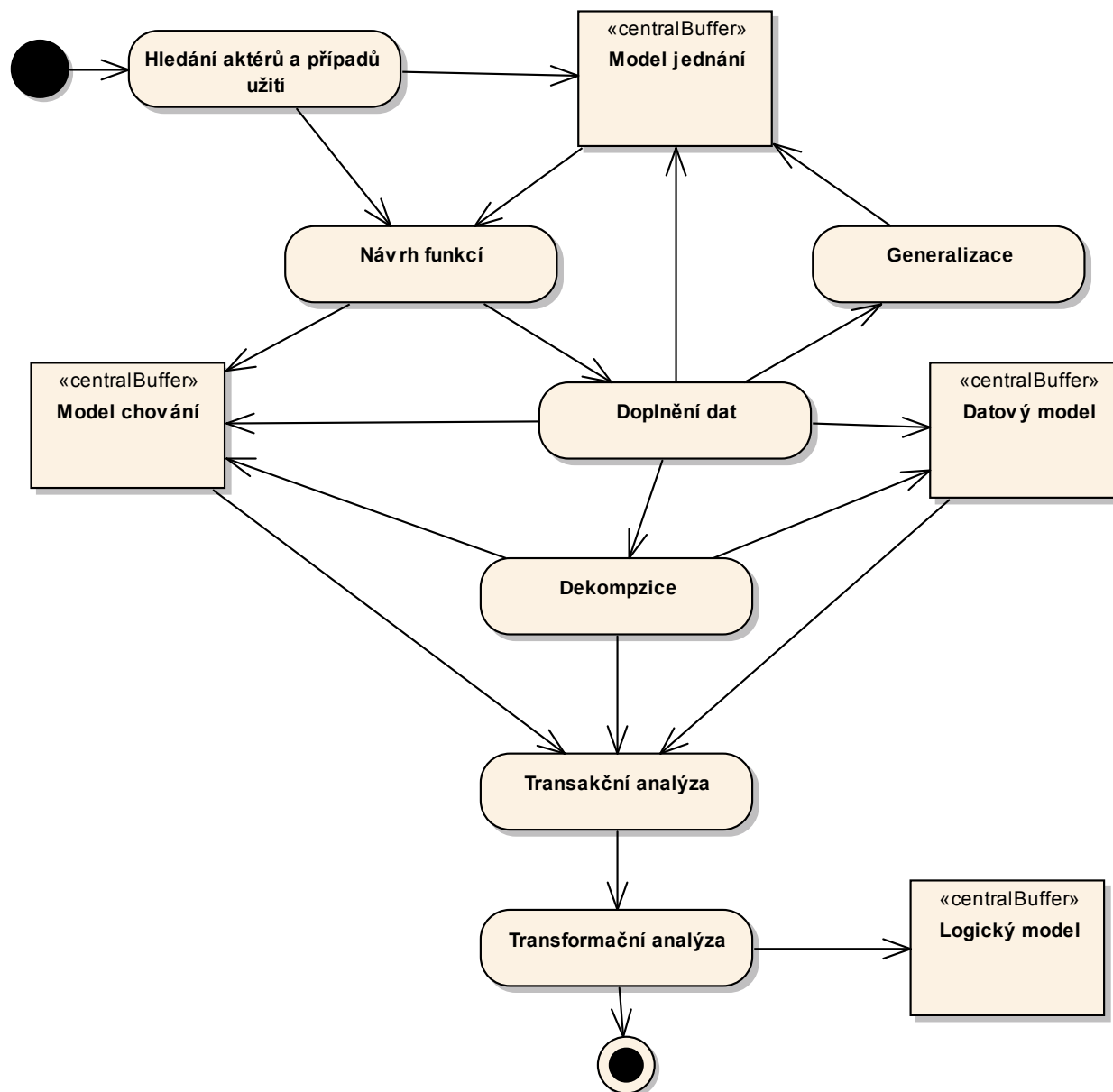


Taxonomie metodik

- **Klasické**
 - Strukturované, objektové, komponentové
 - Vodopád (MSA, SSADM)
 - Unifikovaný proces (UP)
 - RUP (Rational Unified Process)
- **Agilní** (vycházejí z [manifestu](#) agilních metod)
 - Testy řízený vývoj
 - Extrémní programování (XP)
 - Scrum
 - AUP (Agile Unified Process)

Moderní strukturovaná analýza (MSA)

- Autoři E.Yourdon (analýza), M.Page-Jones (návrh).
- Klasický vodopádový model.
- Sběr požadavků – výstupem je odborný článek a hrubý model jednání.
- Analýza (E.Yourdon):
 - Vytváří analytický model, který se skládá z datového modelu a funkční hierarchie – sady diagramů datových toků (diagramů aktivit), popisujících požadovanou funkčnost.
- Návrh (M.Page-Jones):
 - Výstup analýzy podrob tzv. transakční a transformační analýze a navrhni řešení.
- Implementace podle návrhu



MSA: Analýza

- Vstupem je neformální specifikace ([vize projektu](#)).
- Sestavte aktéry a případy užití ([kontext](#)).
- Pro každý případ užití navrhnete funkci ([proces](#)), která bude tento případ obsluhovat.
- Zjistěte potřebná data – pro každou funkci vymysli vstupní a výstupní data.
- Proveďte generalizaci a [dekompozici](#) těchto funkcí.
- Generalizací dospějete k modelu jednání, navrhnete strukturu „menu“ aplikace.
- Dekompozicí postupně dojdete k elementárním funkcím, které již postačí popsat textově (pomocí tzv. minispecifikací).

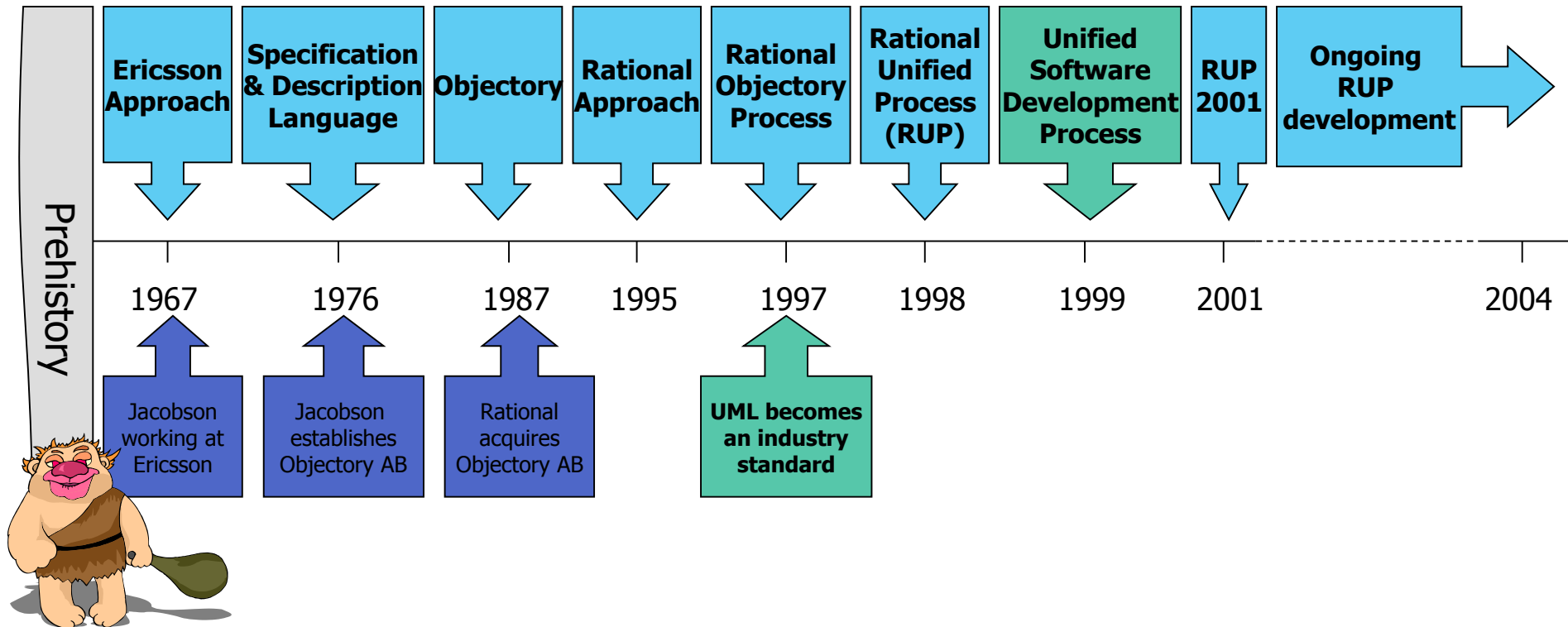
MSA: Návrh

- Proveď **transakční analýzu** - vezmi všechny diagramy datových toků (aktivity) a vyber z nich sadu pro každou transakci (případ užití).
- Pro každý takto získaný popis transakce vysleduj jádro, které transformuje data – tzv. **transformační analýza**.
- Z jádra učiň ředitele této transakce (**řídící třídu**) a doplň komunikaci s třídami získávajícími vstupní data a formátujícími výstupní data.
- K této sadě přidej tzv. **transakční monitor**: modul, který zjistí, o kterou transakci se jedná a vyvolá odpovídající řídící třídu.

Unifikovaný proces vývoje (UP)

- UP je:
 - Průmyslový standard kostry vývojového procesu
 - Využívá notace UML (Unified Modeling Language)
 - Je to otevřený standard (na rozdíl např. od RUP – Rational Unified Process, dříve Rational, nyní IBM)
 - Základem je publikace Jacobson, Booch, Rumbaugh: „The Unified Software Development Process“
- UP je:
 - Řízen požadavky a případy užití (use case driven)
 - Řízen rizikem
 - Staví na architektuře, komponentách a znovupoužití
 - Iterativní a přírůstkový proces
- UP je:
 - pouze generická metodika, musí být uzpůsobena pro danou firmu, projekt - standardy, šablony, nástroje, ...

UP: Historie

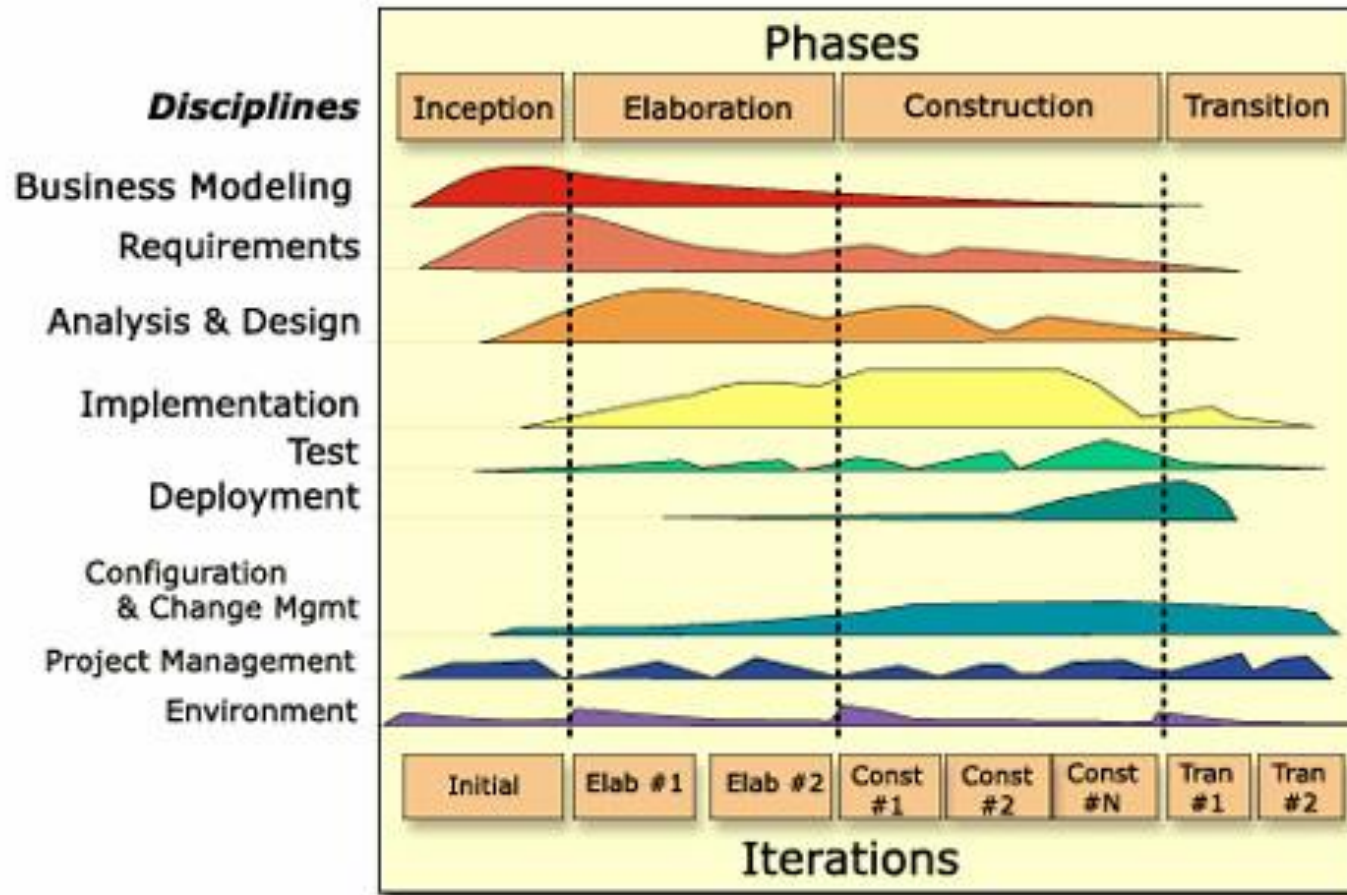


Zdroj: © Arlow, Neustadt: UML 2 a objektově orientovaný návrh.

UP: Iterační a přírůstkový vývoj

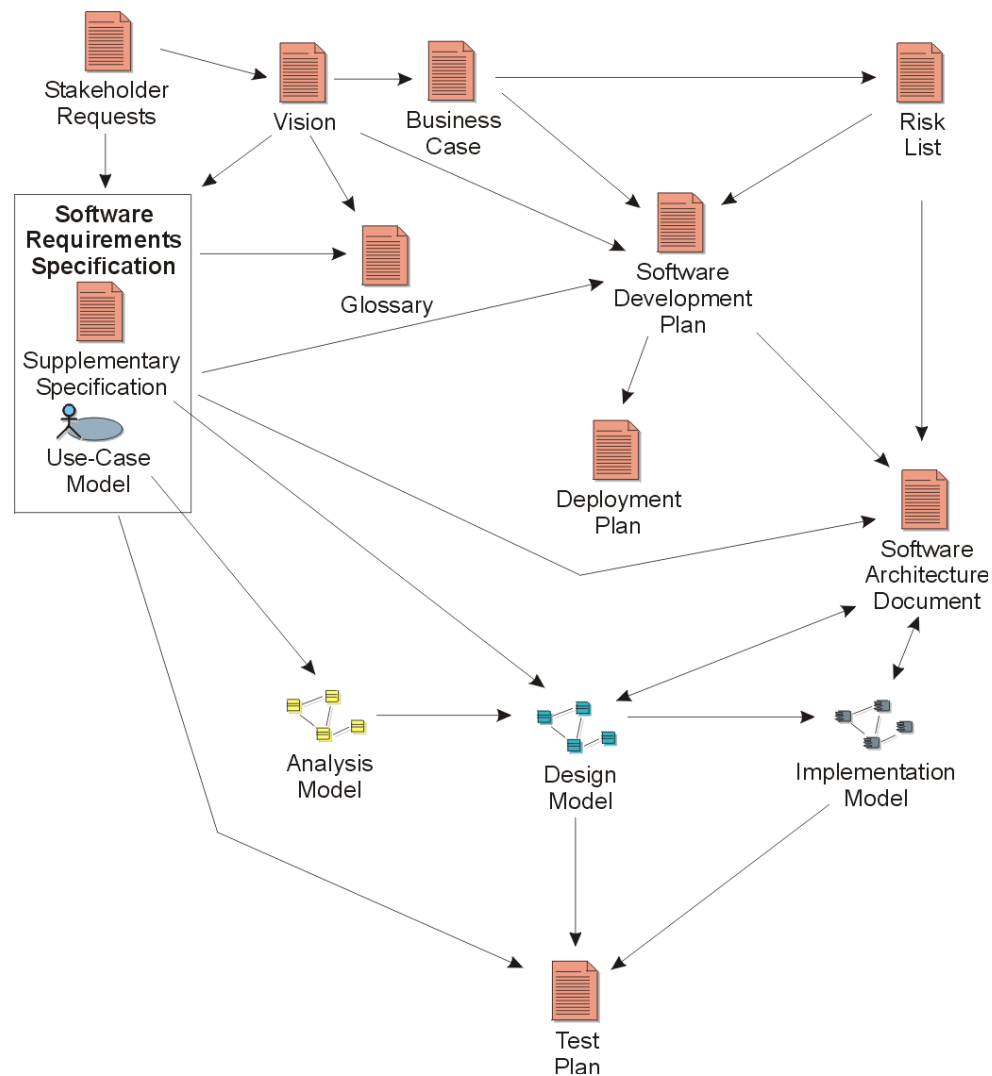
- Základem UP jsou iterace.
- Každá iterace je malý vodopád:
 - Plánování
 - Analýza a návrh
 - Konstrukce, implementace
 - Integrace a testování
 - Uvolnění verze.
- Celý produkt se tvoří řadou iterací, které představují přírůstky k řešení
- Iterace se mohou překrývat (paralelní vývoj)
- Iterace jsou rozděleny do fází, každá fáze může zahrnovat několik iterací, každá fáze končí milníkem

UP: Fáze a discipliny

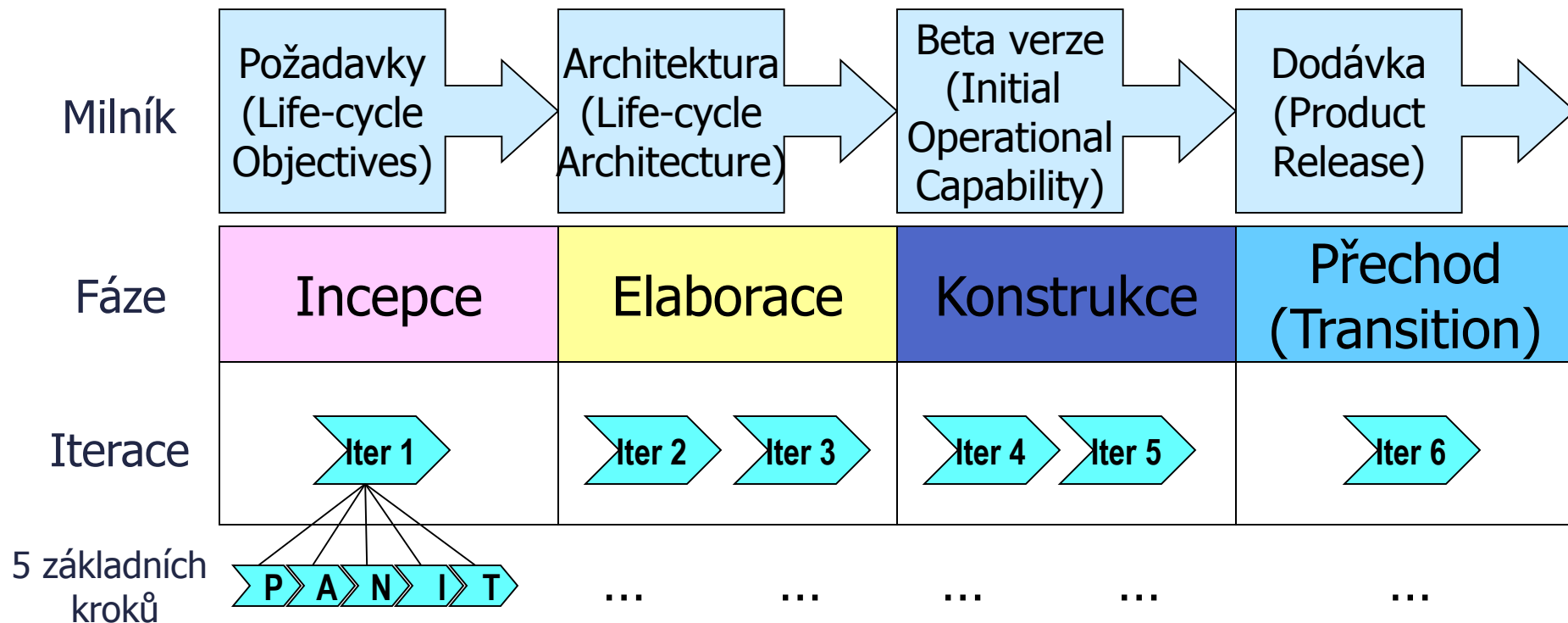


Zdroj: © Jacobson, Booch, Rumbaugh: The Unified Process.

Hlavní artefakty



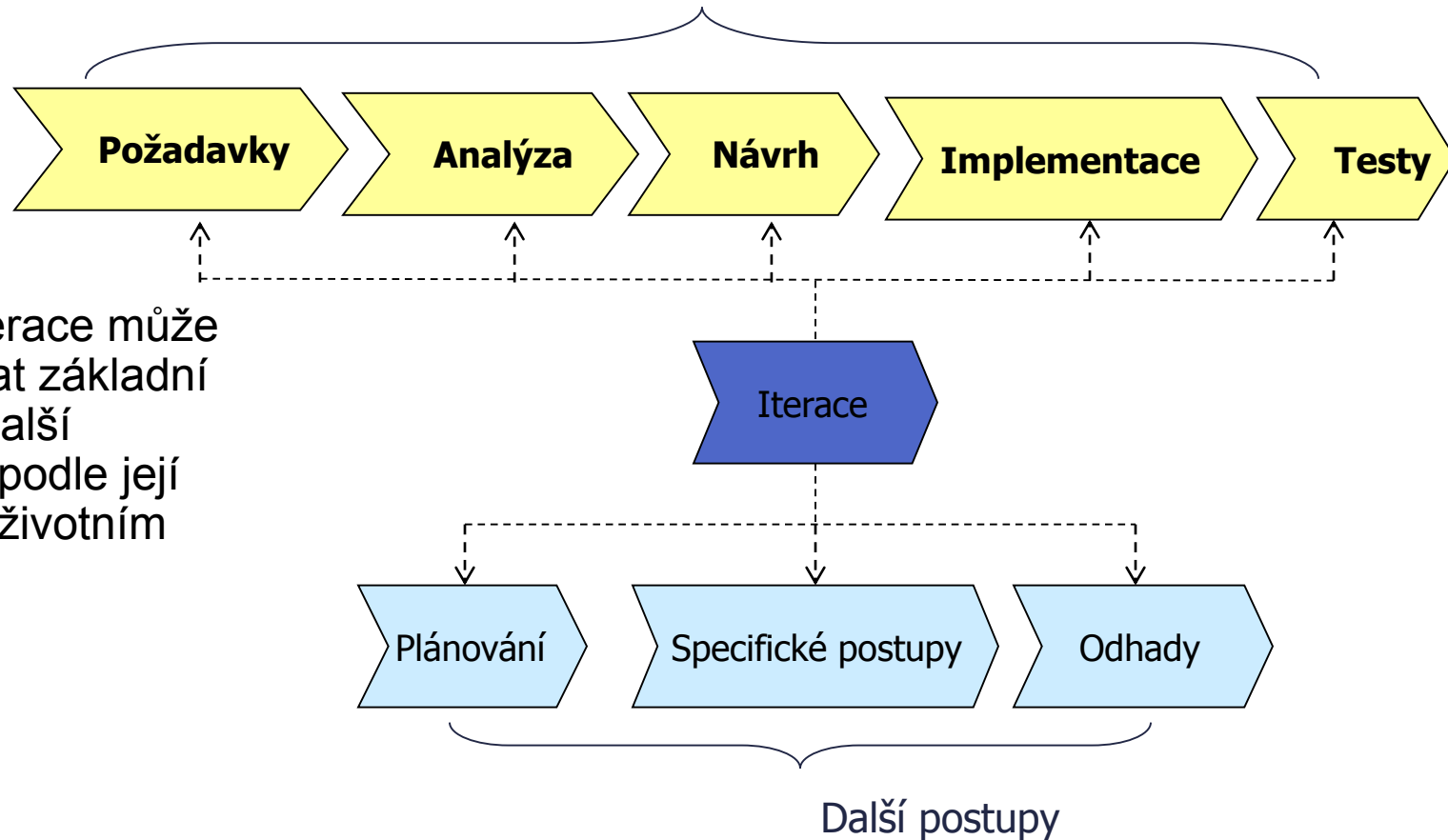
UP: Struktura



- Každá fáze může zahrnovat několik iterací.
 - Skutečný počet závisí na rozsahu projektu, u malých projektů může mít fáze jen jednu iteraci.
- Každá fáze končí milníkem.

Kroky v rámci iterace

UP specifikuje 5 základních kroků

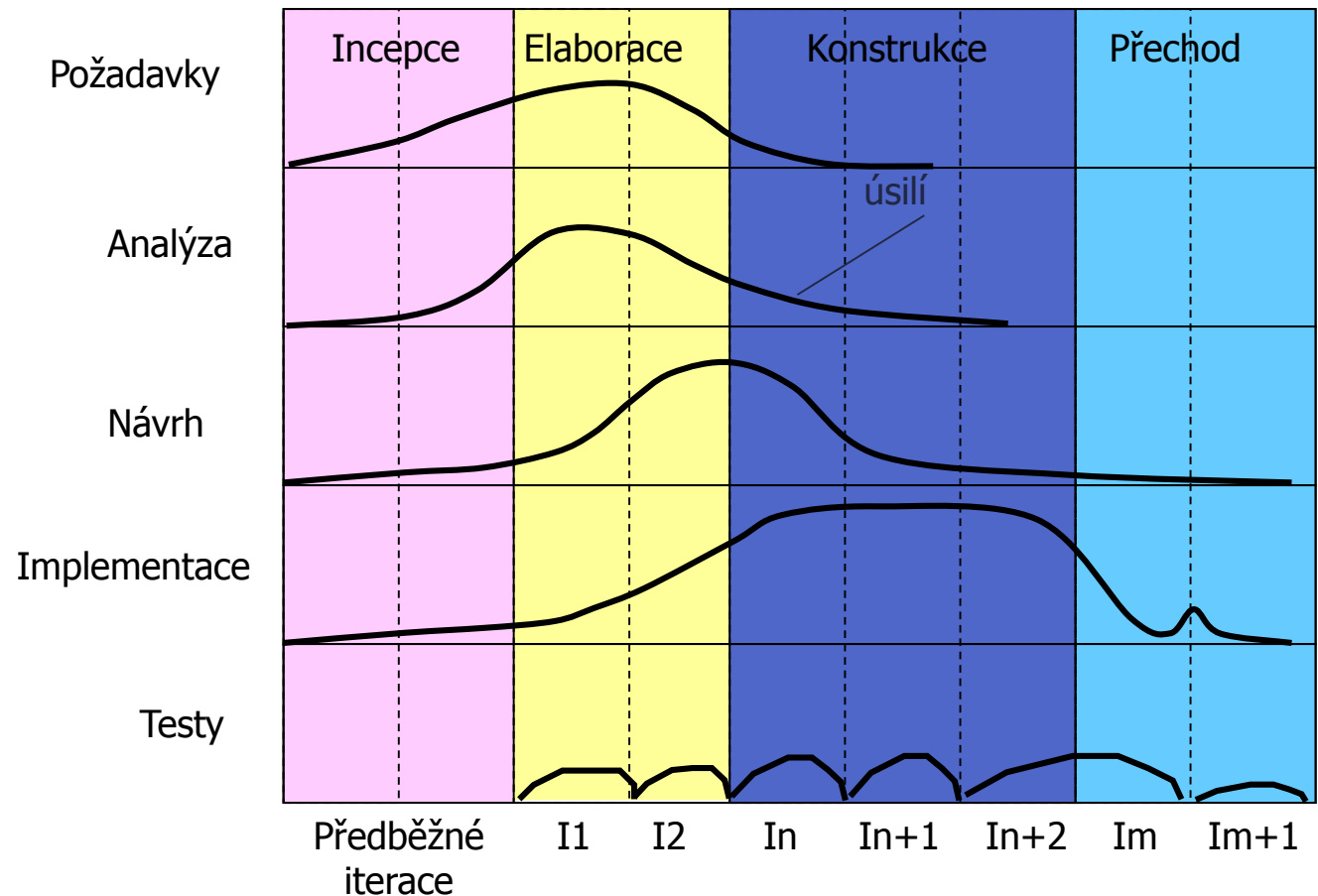


- ◆ Každá iterace může obsahovat základní kroky a další postupy, podle její polohy v životním cyklu.

UP: Úsilí vynaložené ve fázích

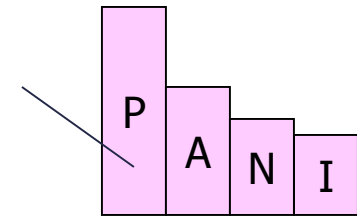
- Pro každou fázi je nutno specifikovat:

- Základní postupy
- Cíl fáze
- Co je milník na konci fáze



UP: Incepce

Relativní úsilí pro jednotlivé kroky



(zahájení, úvodní studie)

Zaměřeno na	Požadavky – sestavit případy užití a vymezit rozsah. Vytipovat jejich důležitost.	Incepce	Elaborace	Konstrukce	Přechod
	Analýza – zhodnotit proveditelnost				
	Návrh – ověření konceptu nebo technický prototyp				
	Implementace – ověření konceptu nebo technický prototyp				
	Testy – zde nemají smysl				
Cíle	Ověřit proveditelnost projektu – případně s využitím prototypů Sestavit požadavky a případy užití Vymezit hranici systému Identifikovat rizika				

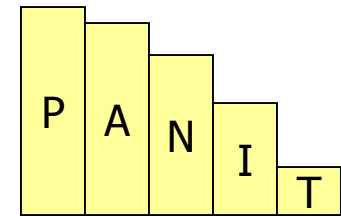
Požadavky

Milník – konec inceptce

- Cíle projektu představující podmínky pro úspěšné řešení:
 - Je určena hranice systému (system scope).
 - Jsou definovány klíčové požadavky na systém a jsou schváleny zainteresovanými osobami (stakeholders).
 - Existuje vize architektury systému, zatím jen hrubě.
 - Je zpracována první analýza rizik.
 - Je zpracován obchodní případ (business case), odhad nákladů a výnosů.
 - Je potvrzena proveditelnost projektu (feasibility study).
 - Zainteresované osoby (stakeholders) souhlasí s objektivními charakteristikami projektu.

UP: Elaborace

(rozbor, analýza)



Zaměřeno na	Požadavky – zkompletovat požadavky a hranici rozsahu projektu	Incepce	Elaborace	Konstrukce	Přechod
	Analýza – vymezení skutečného rozsahu				
	Návrh – vytvoření stabilní architektury				
	Implementace – realizace základny pro architekturu				
	Testy – testování architektonické základny				
Cíle	Vytvoření a návrh architektury Vyhodnocení rizik Výběr 80% funkčních požadavků Vytvoření detailního plánu pro fázi konstrukce Formulace požadavků na zdroje, čas, vybavení, týmy a cenu				

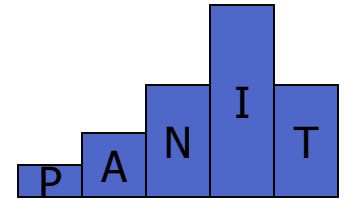
Analýza

Milník – konec elaborace

- Je určena architektura :
 - Je definován robustní proveditelný architektonický základ řešení.
 - Je doplněna analýza rizik.
 - Je vytvořen plán projektu pro realistickou představu o postupu konstrukce a implementace.
 - Obchodní plán je porovnán s plánem a původní představou.
 - Zainteresované osoby (stakeholders) souhlasí s pokračováním.

UP: Konstrukce

(návrh, implementace)



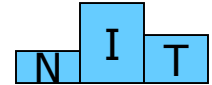
Zaměřeno na	Požadavky – objevování chybějících požadavků	Incepce	Elaborace	Konstrukce	Přechod
	Analýza – dokončení analytického modelu				
	Návrh – dokončení návrhového modelu				
	Implementace – vytvoření počáteční funkčnosti – beta verze				
	Testy – testování beta verze				
Cíle	Dokončení specifikace Dokončení analýzy, návrhu, implementace a testování Údržba integrity architektury systému Vyhodnocení rizik				

Návrh

Milník - konec konstrukce

- Částečná funkčnost systému:
 - Produkt je připraven k beta-testování v prostředí uživatele.

UP: Přechod



(transition, provoz)

Zaměřeno na	Požadavky – nic	Incepce	Elaborace	Konstrukce	Přechod
	Analýza – nic				
	Návrh – modifikace podle výsledků beta testů				
	Implementace – přizpůsobení softwaru prostředí uživatele, opravy chyb z beta testů				
	Testy – akceptační testy v prostředí uživatele				
Cíle	Opravy chyb Příprava prostředí uživatele a uzpůsobení softwaru tomuto prostředí Modifikace dle odhalených problémů Tvorba příruček a manuálů, dokumentace Poskytování konzultační podpory uživateli Vyhodnocení projektu				

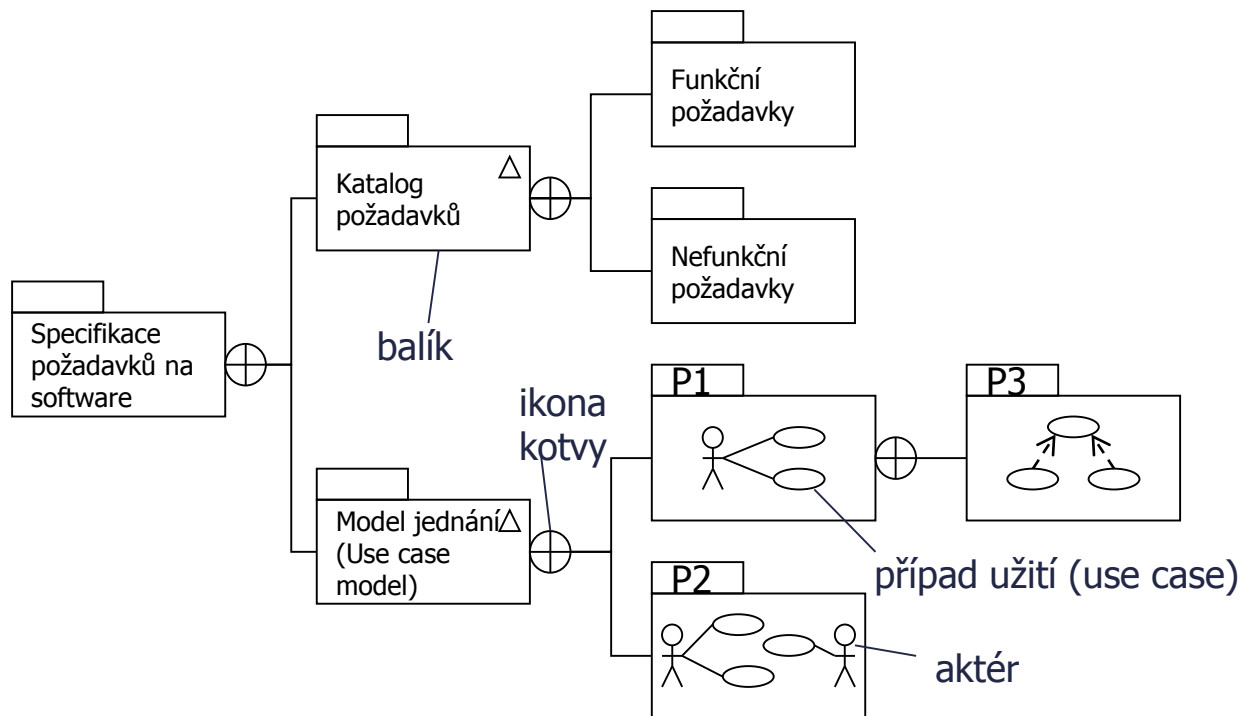
Implementace

Milník – konec přechodu

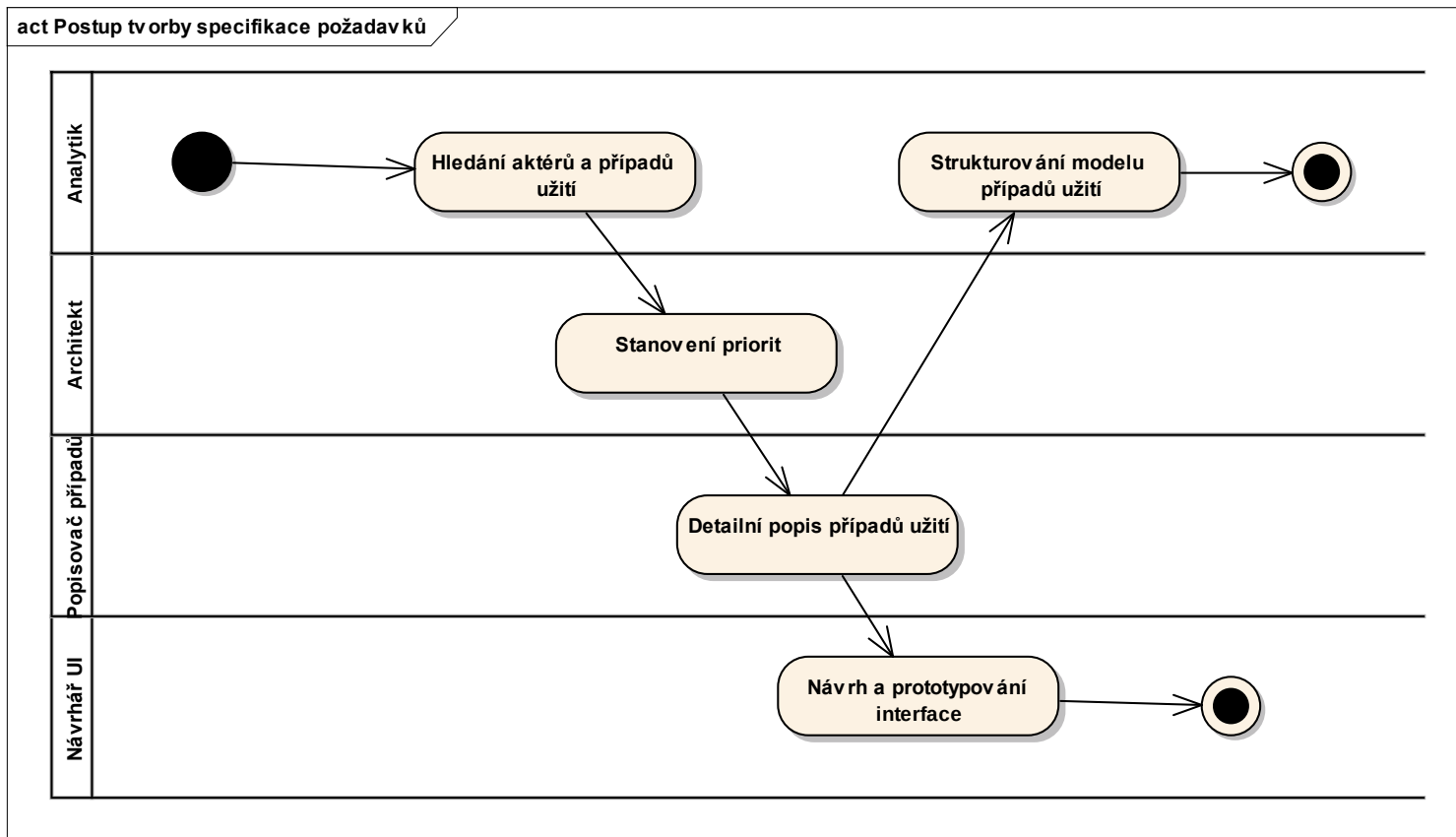
- Produkt je k dispozici:
 - Proběhlo beta-testování, akceptační testy a jsou opraveny nalezené chyby.
 - Produkt je uvolněn pro uživatele.

Požadavky a jejich metamodel

- Smyslem tohoto kroku je vytvořit přehledovou („high-level“) specifikaci toho, co se má implementovat.
- Sháníme materiály, děláme interview se zainteresovanými osobami („stakeholders“), abychom zjistili, co by od systému potřebovali – jaké jsou jejich požadavky.

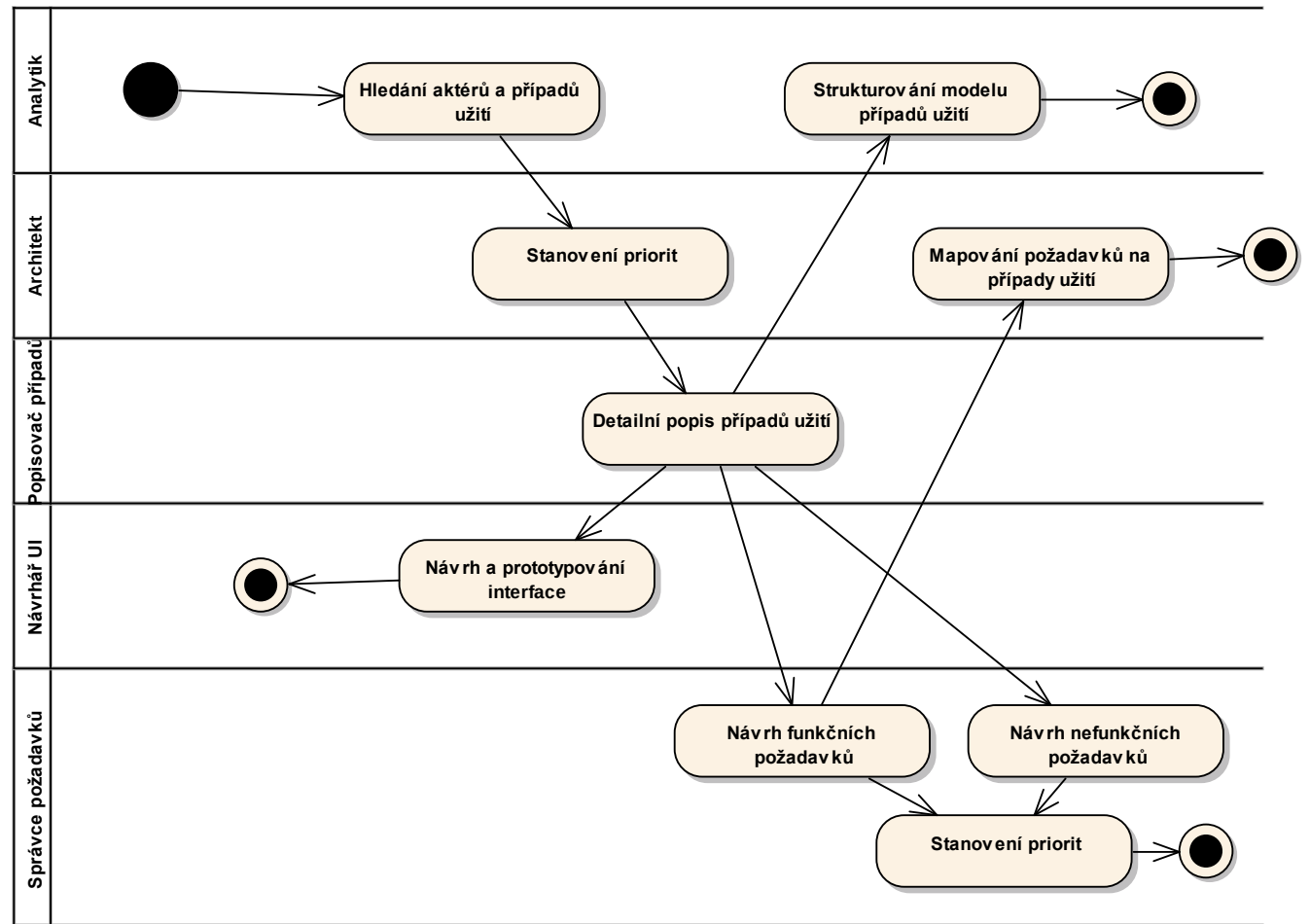


Postup získávání požadavků



A navíc

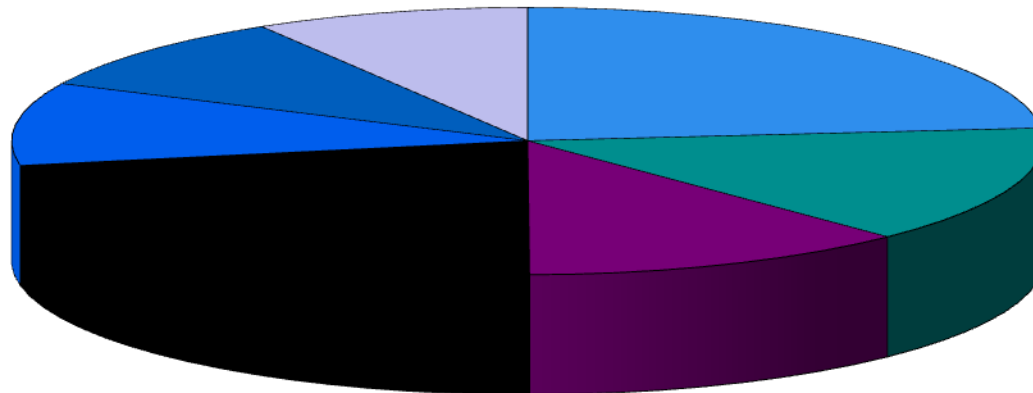
act Postup tvorby specifikace požadavků - 2



Pro správnou správu požadavků je třeba přidat aktivity související s návrhem funkčních a nefunkčních požadavků a jejich zapracováním do modelu.

Důležitost požadavků

Zdroje chyb projektů



- Neúplné požadavky
- Uživatel nebyl dostatečně zainteresován
- Nedostatek zdrojů
- Nerealistická očekávání
- Chybí podpora exekutivy
- Změny požadavků
- Nedostatek plánování
- Není dále zapotřebí

Neúplná specifikace požadavků představuje primární zdroj možného neúspěchu projektu!

Zdroj: Standish Group, "The CHAOS Report (1994) "

Co to jsou požadavky?

- Požadavky - “Specifikace toho, co by mělo být implementováno”:
 - Jaké chování bude systém nabízet.
 - Specifické vlastnosti systému.
 - Jaká omezení je třeba brát v úvahu.
- V metodice UP se vytváří specifikace požadavků (**Software Requirements Specification - SRS**):
 - Na počátku procesu konstrukce softwaru je dohoda o požadavcích mezi všemi zainteresovanými stranami.
 - Katalog požadavků by měl být rozumně organizován do částí, které spolu souvisí.
- SRS obsahuje dvě části:
 - **Katalog (model) požadavků**, kam patří funkční a nefunkční požadavky.
 - **Model jednání (use case model)**, kam patří přehled aktérů a případů užití (use cases).

Jak požadavky zapisovat?

<id> The <system> **shall** <function>

Unikátní identifikátor

Jméno systému

keyword

Požadavek

Např. "32 Bankomat **by měl** validovat PIN."

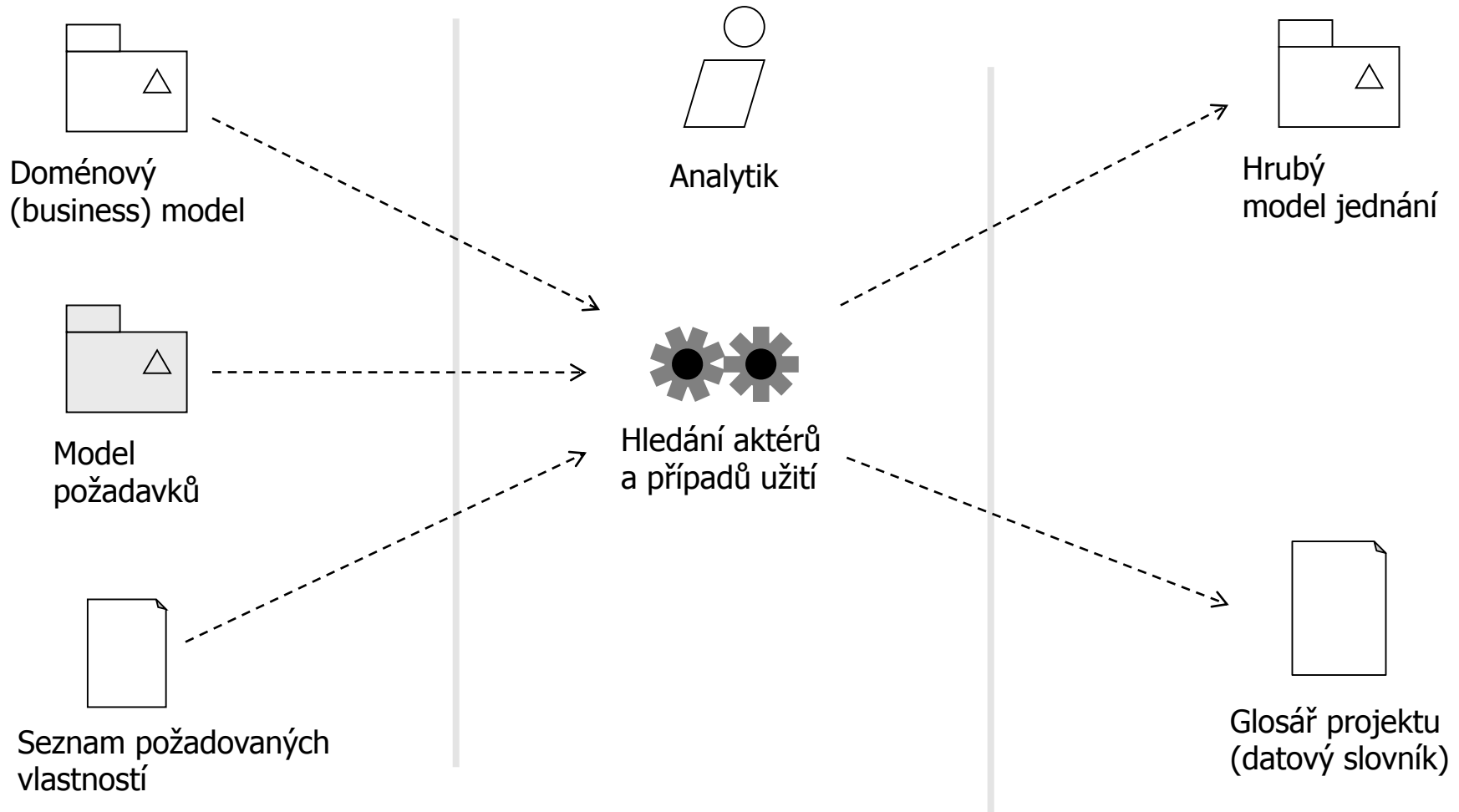
- **Není žádný obecný standard na psaní požadavků!**
 - Doporučuje se např. formát výše uvedený.
- **Funkční požadavky – co by měl systém dělat:**
 - „Bankomat by měl poskytovat možnost identifikace zákazníka.“
- **Nefunkční požadavky – omezení na způsob konstrukce a implementace:**
 - „Bankomat by měl identifikaci zákazníka zvládnout vždy nejdéle za 4 sec.“

Modelování případů užití

- Součástí modelování a správy požadavků by mělo být i modelování případů užití.
- Modelování případů užití spočívá v hledání:
 - hranice systému
 - aktérů,
 - případů užití (use cases)
 - specifikace případů a
 - scénářů, které se za nimi skrývají – strukturovaný textový popis průběhu případů užití.
- To nám umožní definovat rozsah systému, kdo a na co jej má používat.

Hledání aktérů a případů užití

Zdroj: © Jacobson, Booch, Rumbaugh: The Unified Process.



Subjekt, aktéři, případy užití

- Předtím, než budeme cokoliv vytvářet, měli bychom znát:
 - kde leží hranice systému,
 - kdo nebo co bude systém užívat,
 - jaké služby bude systém uživatelům nabízet.
- Do modelu jednání vložíme:
 - Subjekt – hranici systému
 - Aktéry – kdo se systémem komunikuje
 - Případy užití – na co jej používá
 - Vztahy mezi aktéry a případy užití.

subjekt

JménoSystému

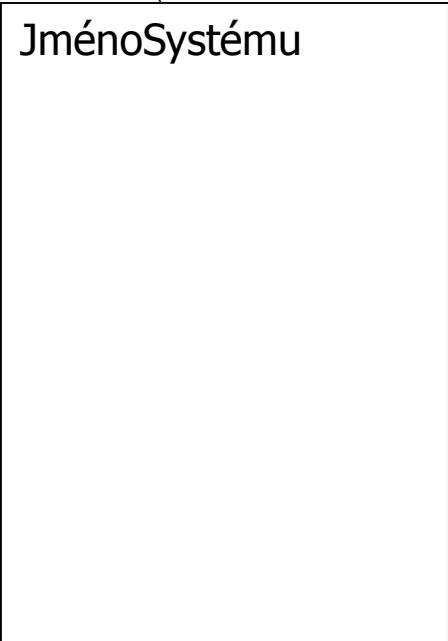
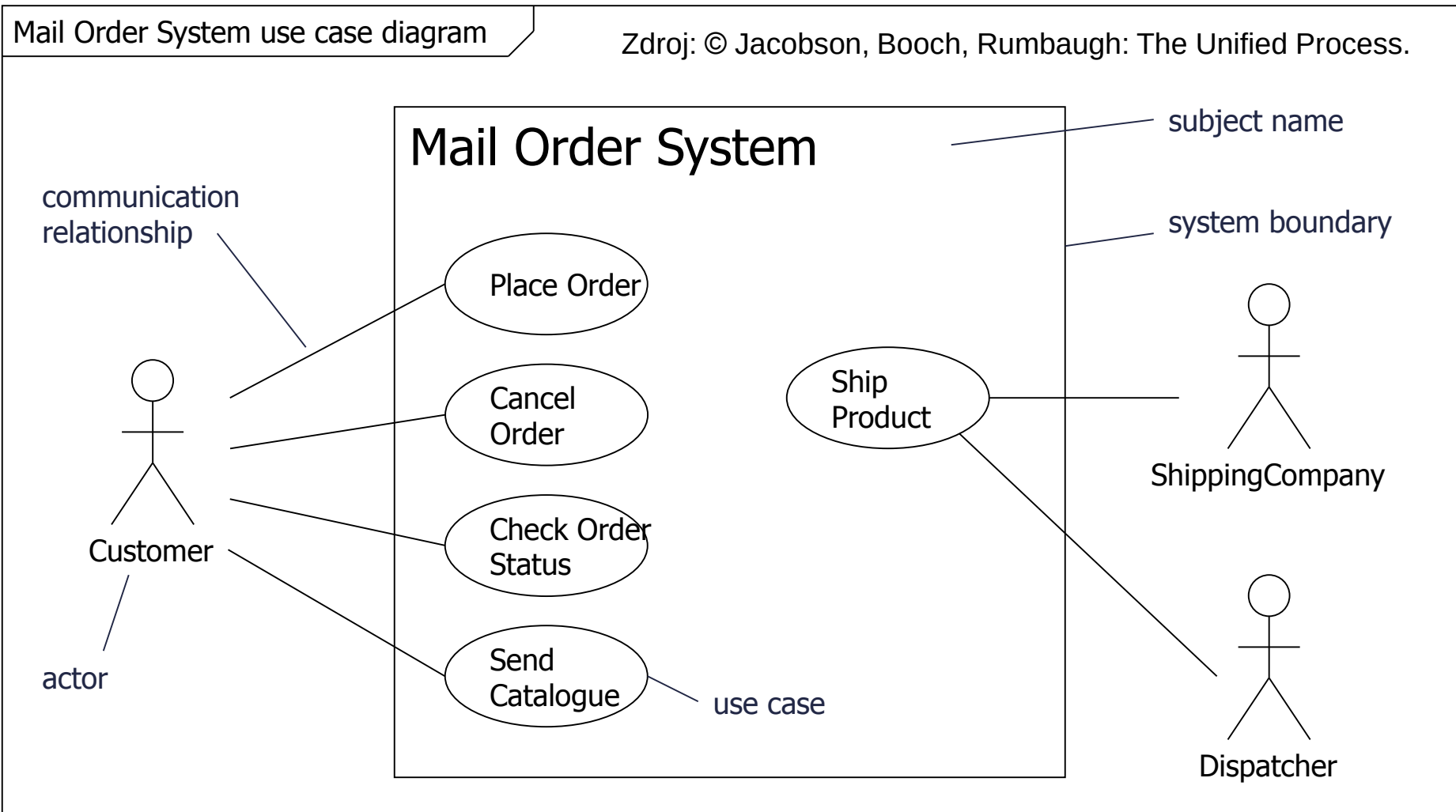


Diagram případů užití



Glosář projektu (Data Dictionary)

Project Glossary

Termín1

Definice
Synonyma
Homonyma

Termín2

Definice
Synonyma
Homonyma

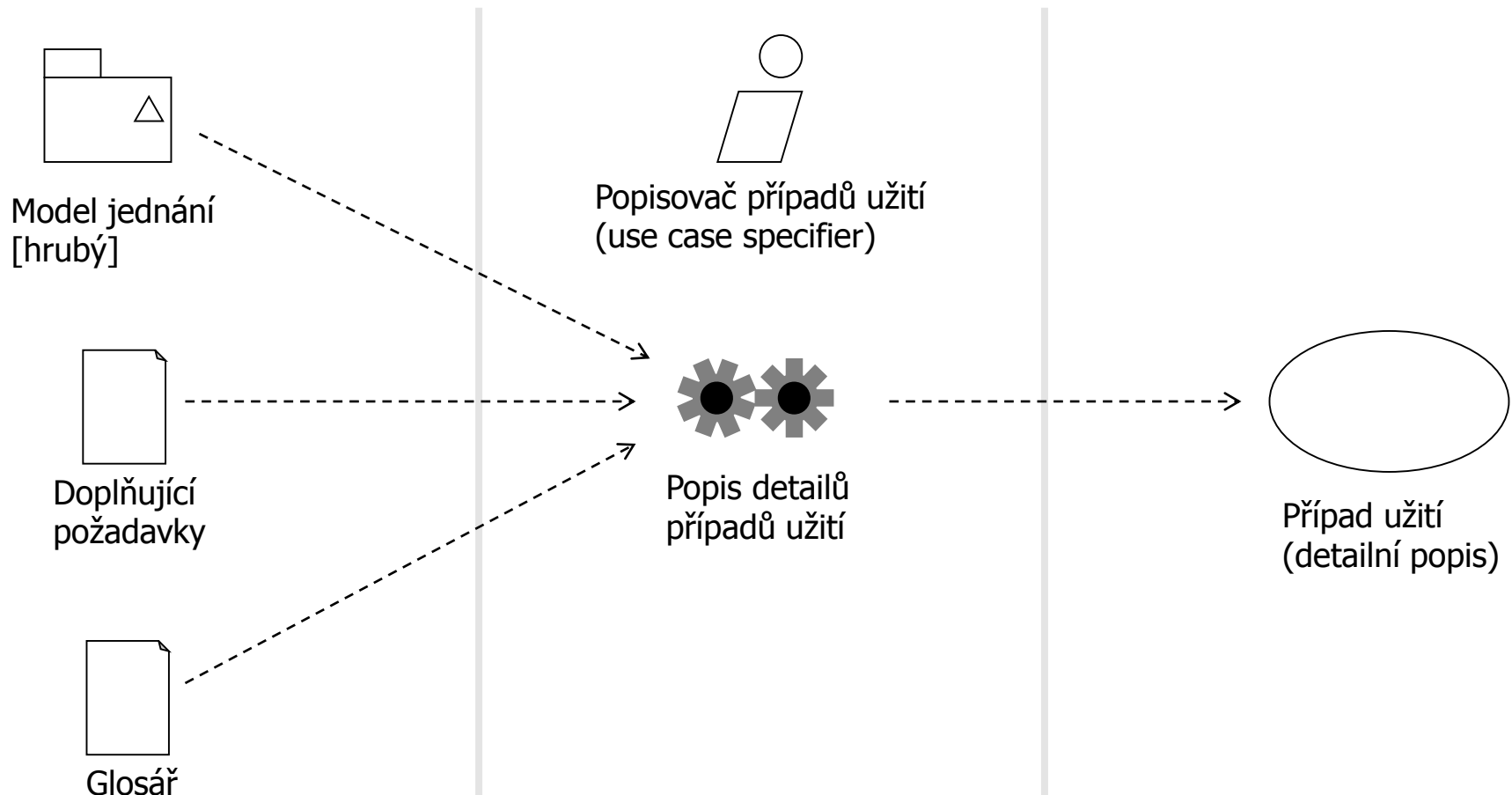
Termín3

Definice
Synonyma
Homonyma

...

- V každé doméně existuje žargon, proto doplňujeme modely glosářem.
- Smyslem glosáře je definovat synonyma a homonyma.
- Vytváříte slovník pro potřeby diskuse se zainteresovanými osobami, které mohou používat žargon dané domény.

Detailní popis případů užití



Zdroj: © Jacobson, Booch, Rumbaugh: The Unified Process.

Detailní specifikace případů užití

Název případu užití	{	Use case: PaySalesTax
Jeho identifikátor	{	ID: 1
Stručný popis	{	Brief description: Pay Sales Tax to the Tax Authority at the end of the business quarter.
Aktéři participující na tomto případě užití	{	Primary actors: Time
	{	Secondary actors: TaxAuthority
Podmínka pro stav systému před nastartováním případu užití	{	Preconditions: 1. It is the end of the business quarter.
Scénář případu užití jako poslupnost kroků	{	Main flow: / implicit time actor 1. The use case starts when it is the end of the business quarter. 2. The system determines the amount of Sales Tax owed to the Tax Authority. 3. The system sends an electronic payment to the Tax Authority.
Stav systému poté, co případ užití skončí	{	Postconditions: 1. The Tax Authority receives the correct amount of Sales Tax.
Alternativy	{	Alternative flows: None.

Předpoklady a důsledky

- Předpoklady (preconditions) a důsledky (postconditions) představují omezení.
- Předpoklady omezují stav systému před zahájením scénáře.
- Důsledky omezují stav systému po ukončení scénáře.
- Pokud nejsou žádné předpoklady ani důsledky, je třeba to zdůraznit např. klíčovým slovem "None" pod hlavičkou.

Place Order

Preconditions:

1. A valid user has logged on to the system

Postconditions:

1. The order has been marked confirmed and is saved by the system

Hlavní scénář

<číslo> <někdo> <nějaká akce>

- Scénář zachycuje posloupnost kroků v rámci zpracování případu užití.
- Vždy se začíná tím že aktér něco provede (iniciuje událost).
 - Dobrá technika je zahájit scénář:
 - 1) Případ užití se spustí, když aktér - <funkce>
- Kroky scénáře by měly být :
 - deklarativní,
 - očíslované,
 - Uspořádané v čase.
- Hlavní scénář by měl představovat úspěšné použití:
 - Vše probíhá podle očekávání, nenastaly chyby, odchylky, přerušení apod.
 - Alternativní scénáře pak popisují tyto situace.

Větvení: If

- Pro vyjádření možnosti větvení používáme klíčové slovo **if** -
 - Po něm musí bezprostředně následovat logický výraz (typu Boolean)
- Využívejte odsazování a číslování pro podmíněné složky scénářů.
- Lze využít větev označenou **else**, abychom vyjádřili, co se stane, když podmínka nebude splněna (viz dále).

Use case: ManageBasket
ID: 2
Brief description: The Customer changes the quantity of an item in the basket.
Primary actors: Customer
Secondary actors: None.
Preconditions: 1. The shopping basket contents are visible.
Main flow: 1. The use case starts when the Customer selects an item in the basket. 2. If the Customer selects "delete item" 2.1 The system removes the item from the basket. 3. If the Customer types in a new quantity 3.1 The system updates the quantity of the item in the basket.
Postconditions: None.
Alternative flows: None.

Opakování: For

- Pro vyjádření možnosti opakování můžeme použít klíčové slovo **for**.
- Výraz na klíčovým slovem **for** indikuje způsobu počet opakování těla cyklu – odsazeného textu za **for**.

Use case: FindProduct
ID: 3
Brief description: The system finds some products based on Customer search criteria and displays them to the Customer.
Actors: Customer
Preconditions: None.
Main flow: 1. The use case starts when the Customer selects "find product". 2. The system asks the Customer for search criteria. 3. The Customer enters the requested criteria. 4. The system searches for products that match the Customer's criteria. 5. If the system finds some matching products then 5.1 For each product found 5.1.1. The system displays a thumbnail sketch of the product. 5.1.2. The system displays a summary of the product details. 5.1.3. The system displays the product price. 6. Else 6.1. The system tells the Customer that no matching products could be found.
Postconditions: None.
Alternative flows: None.

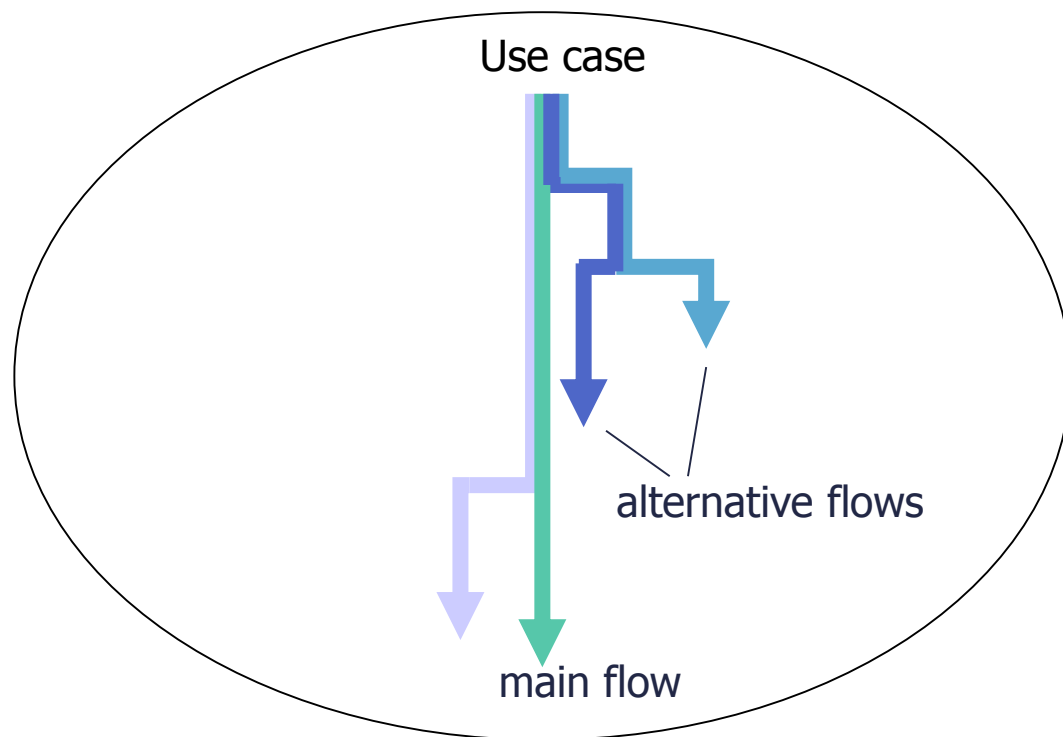
Opakování: While

- Pro vyjádření možnosti opakování můžeme také použít klíčové slovo **while**, pokud se má tělo cyklu opakovat do splnění nějaké podmínky – ta je uvedena v hlavičce cyklu.

Use case: FindProduct
ID: 3
Brief description: The system finds some products based on Customer search criteria and displays them to the Customer.
Primary actors: Customer
Secondary actors: None
Preconditions: None.
Main flow: 1. The use case starts when the Customer selects "find product". 2. The system asks the Customer for search criteria. 3. The Customer enters the requested criteria. 4. The system searches for products that match the Customer's criteria. 5. While the system finds some matching then 5.1 Select the first matching product found 5.2 The system displays a thumbnail sketch of the product. 5.3 The system displays a summary of the product details. 5.4 The system displays the product price. 5.5 The system will ask user for continuation. 6. The system tells the Customer that no matching products could be found.
Postconditions: None.
Alternative flows: None.

Alternativy

- **Pro každý případ užití můžeme specifikovat jeden či více alternativních scénářů:**
 - Alternativní scénáře popisují chybové situace.
 - Alternativní scénáře se nikdy nevrací do hlavního scénáře (na to je if)
- **Potenciálně existuje mnoho alternativních scénářů, měli bychom zachytit:**
 - Nejdůležitější alternativy a dokumenty.
 - Pokud existuje skupina podobných alternativních scénářů, stačí jeden s vysvětlením odlišností.



Postačí dokumentovat jen tolik alternativ, kolik je důležité pro pochopení požadavků!

Referencování alternativ

- Seznam alternativních scénářů se uvádí na konci popisu případu užití.
- Při hledání možných alternativ procházej hlavní scénář a zamysli se nad:
 - alternativami,
 - výjimkami,
 - chybami,
 - možnostmi přerušení.

Alternativní
scénáře

Use case: CreateNewCustomerAccount
ID: 5
Brief description: The system creates a new account for the Customer.
Primary actors: Customer
Secondary actors: None.
Preconditions: None.
Main flow: 1. The use case begins when the Customer selects "create new customer account". 2. While the Customer details are invalid 2.1. The system asks the Customer to enter his or her details comprising email address, password and password again for confirmation. 2.2 The system validates the Customer details. 3. The system creates a new account for the Customer.
Postconditions: 1. A new account has been created for the Customer.
Alternative flows: InvalidEmailAddress InvalidPassword Cancel

Příklady alternativ

Povšimněte si, jak se označují a číslují alternativní scénáře



Alternative flow: CreateNewCustomerAccount:InvalidEmailAddress
ID: 5.1
Brief description: The system informs the Customer that they have entered an invalid email address.
Primary actors: Customer
Secondary actors: None.
Preconditions: 1. The Customer has entered an invalid email address
Alternative flow: 1. The alternative flow begins after step 2.2. of the main flow. 2. The system informs the Customer that he or she entered an invalid email address.
Postconditions: None.

Vždy indikujte, jak alternativní scénář začal. V tomto případě začal po kroku 2.2 v hlavním scénáři



- Alternativní scénář může být spuštěn aktérem namísto hlavního
- Alternativní scénář může být spuštěn po některém kroku hlavního - **after**
- Alternativní scénář může být spuštěn kdykoliv během hlavního - **at any time**

Trasování požadavků

- Pro porovnání funkčních požadavků z katalogu a případů užití z modelu jednání se může hodit matice, kde řádky odpovídají požadavkům, sloupce případům užití.
- Vztah (M:N) mezi nimi vyznačíme v odpovídajícím políčku :
 - Pokud případ užití pokrývá nějaký požadavek.
 - Požadavek může být realizován několika případy užití.
- Nejlépe je použít pro toto trasování nějaký CASE nástroj, kterým odpovídající matici vytvoříme.
- Pokud nemáme žádný CASE nástroj, můžeme vytvořit tuto matici trasování požadavků (Requirements Traceability Matrix) ručně.

		Případy užití			
Požadavky		U1	U2	U3	U4
	R1				
	R2				
	R3				
	R4				
	R5				

Matice trasování požadavků
(Requirements Traceability Matrix)

Kdy používat analýzu případů užití?

- Případy užití popisují systém z pohledu aktérů. To je vhodné:
 - Když u systému dominují funkční požadavky.
 - Když systém poskytuje různou funkcionalitu různým aktérům.
 - Pokud má systém rozmanitá rozhraní.
- Případy užití specifikují chování z hlediska poskytovaných funkcí. Nejsou vhodné:
 - Když u systému dominují nefunkční požadavky.
 - Pokud má systém jen málo aktérů.
 - Pokud má systém jednoduchá rozhraní.

UP: Dodávky a přírůstky

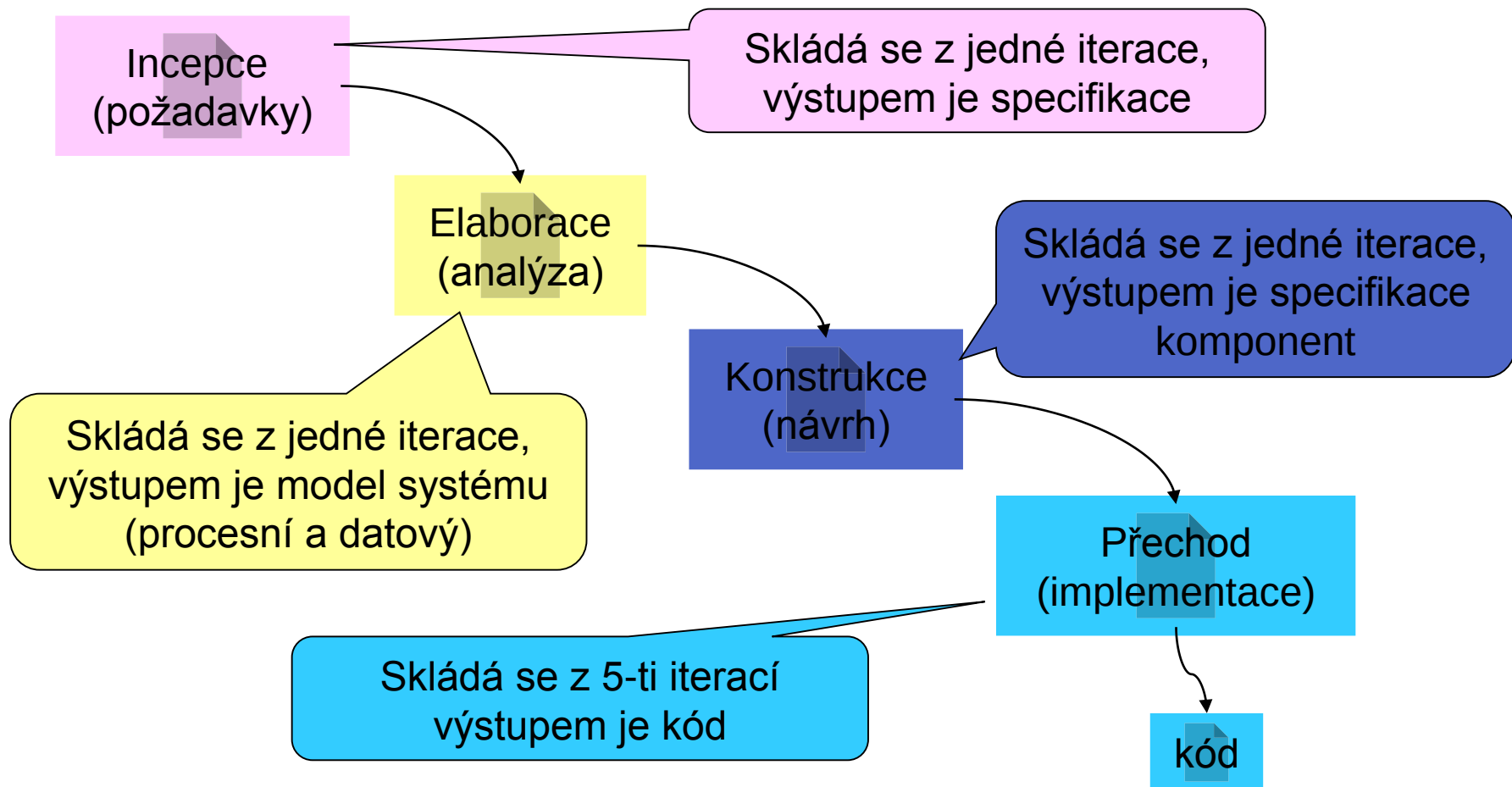
- Každá **iterace** produkuje dodávku.
- **Dodávka** je sada revidovaných a ověřených artefaktů, které:
 - Představují dohodnutý základ pro další vývoj
 - Mohou být měněny jen na základě formalizovaných postupů – konfigurace a změnové řízení.
- **Přírůstek** je rozdíl mezi dvěma následujícími dodávkami.
 - Proto se UP nazývá „iterativní a přírůstkové“.

UP: Shrnutí

- **Iterativní proces:** vodopádový model před sebou tlačí (často dosud netušená) rizika, jejichž odstranění je postupně čím dále dražší. Naproti tomu v iterativním a inkrementálním přístupu dochází k detekci rizik průběžně.
- **Správa a řízení požadavků:** požadavky jsou typicky dynamické a je nutné počítat s tím, že se v čase mění (syndrom *IKIWISI* – *I Will Know It When I See It* – „budu to vědět, až to uvidím“: s nadsázkou vyjádřená neschopnost zákazníka specifikovat dopředu všechny požadavky na vyvíjený produkt).
- **Použití komponentové architektury:** znovupoužití znamená podstatnou úsporu zdrojů.
- **Vizuální modelování softwaru:** model je zjednodušením reality; je vytvářen za účelem dokonalejšího porozumění systému, používá se standardní modelovací jazyk UML.
- **Průběžné zajišťování a ověřování kvality:** nalezení a odstranění problému je 100krát až 1000krát dražší po předání produktu než v úvodních fázích vývoje.
- **Řízení změn:** neřízené změny vedou k chaosu.

Obecná pravidla UP (Best Practices)

Př. Postup dle UP - Anketa



Zdroj: © Běhálek: Grant FRVŠ <http://www.cs.vsb.cz/behalek/frvs/2005/>.

The End